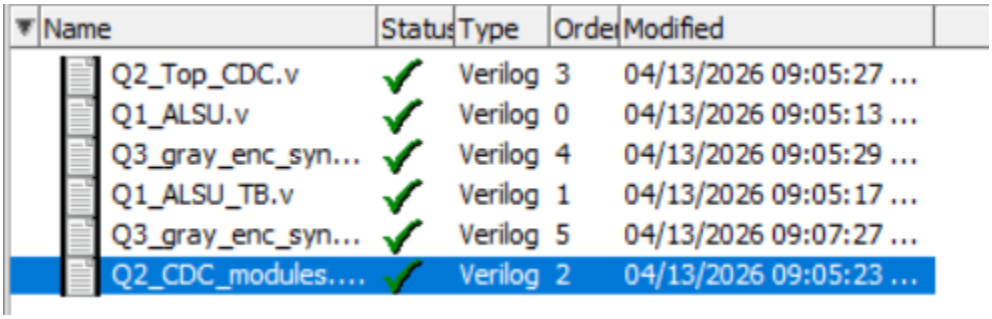
Task5

Esraa Ashraf



Name	Status	Type	Order	Modified
Q2_Top_CDC.v	✓	Verilog	3	04/13/2026 09:05:27 ...
Q1_ALSU.v	✓	Verilog	0	04/13/2026 09:05:13 ...
Q3_gray_enc_syn...	✓	Verilog	4	04/13/2026 09:05:29 ...
Q1_ALSU_TB.v	✓	Verilog	1	04/13/2026 09:05:17 ...
Q3_gray_enc_syn...	✓	Verilog	5	04/13/2026 09:07:27 ...
Q2_CDC_modules....	✓	Verilog	2	04/13/2026 09:05:23 ...

Q1-

```
`timescale 1ns/1ps

module ALSU #(
    parameter INPUT_PRIORITY = "A",
    parameter FULL_ADDER     = "ON"
)(
    input  wire        clk, rst,
    input  wire [2:0]  A, B, opcode,
    input  wire        cin, serial_in,
    input  wire        red_op_A, red_op_B,
    input  wire        bypass_A, bypass_B,
    input  wire        direction,
    output reg  [5:0]  out,
    output reg  [15:0] leds
);

    // input registers (DFF for every input except clk and rst)
    reg [2:0] A_r, B_r, opcode_r;
    reg       cin_r, serial_r, red_A_r, red_B_r;
    reg       byp_A_r, byp_B_r, dir_r;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            A_r <= 0; B_r <= 0; opcode_r <= 0;
```

```
            cin_r <= 0; serial_r <= 0;
            red_A_r <= 0; red_B_r <= 0;
            byp_A_r <= 0; byp_B_r <= 0; dir_r <= 0;
        end else begin
            A_r <= A; B_r <= B; opcode_r <= opcode;
            cin_r <= cin; serial_r <= serial_in;
            red_A_r <= red_op_A; red_B_r <= red_op_B;
            byp_A_r <= bypass_A; byp_B_r <= bypass_B;
            dir_r <= direction;
        end
    end

    // invalid when opcode = 110 or 111, or reduction used with non AND/XOR
opcode
    wire inv_opcode = opcode_r[2] & opcode_r[1];
    wire inv_red    = (red_A_r | red_B_r) & (opcode_r[2] | opcode_r[1]);
    wire invalid    = inv_opcode | inv_red;

    always @(posedge clk or posedge rst) begin
        if (rst)          leds <= 0;
        else if (invalid) leds <= ~leds;
        else              leds <= 0;
    end

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            out <= 0;
        end else begin
            if (byp_A_r && byp_B_r)
                out <= (INPUT_PRIORITY == "A") ? {3'b0, A_r} : {3'b0, B_r};
            else if (byp_A_r)
                out <= {3'b0, A_r};
            else if (byp_B_r)
                out <= {3'b0, B_r};
            else if (invalid)
                out <= 0;
            else begin
                case (opcode_r)
                    3'b000: begin
                        if (red_A_r && red_B_r)
                            out <= (INPUT_PRIORITY == "A") ? {5'b0, &A_r} :
{5'b0, &B_r};
                        else if (red_A_r) out <= {5'b0, &A_r};
                        else if (red_B_r) out <= {5'b0, &B_r};
                        else              out <= {3'b0, A_r & B_r};
```

```
                    end
                    3'b001: begin
                        if (red_A_r && red_B_r)
                            out <= (INPUT_PRIORITY == "A") ? {5'b0, ^A_r} :
{5'b0, ^B_r};
                        else if (red_A_r) out <= {5'b0, ^A_r};
                        else if (red_B_r) out <= {5'b0, ^B_r};
                        else              out <= {3'b0, A_r ^ B_r};
                    end
                    3'b010: begin
                        if (FULL_ADDER == "ON") out <= A_r + B_r + cin_r;
                        else                    out <= A_r + B_r;
                    end
                    3'b011: out <= A_r * B_r;
                    3'b100: begin
                        if (dir_r) out <= {out[4:0], serial_r};
                        else       out <= {serial_r, out[5:1]};
                    end
                    3'b101: begin
                        if (dir_r) out <= {out[4:0], out[5]};
                        else       out <= {out[0], out[5:1]};
                    end
                    default: out <= 0;
                endcase
            end
        end
    end

endmodule
```

tb-

```
`timescale 1ns/1ps


module ALSU_TB;

    reg        clk, rst;
    reg [2:0]  A, B, opcode;
    reg        cin, serial_in, red_op_A, red_op_B;
```

```
    reg        bypass_A, bypass_B, direction;
    wire [5:0]  out;
    wire [15:0] leds;

    ALSU DUT (
        .clk(clk), .rst(rst),
        .A(A), .B(B), .opcode(opcode),
        .cin(cin), .serial_in(serial_in),
        .red_op_A(red_op_A), .red_op_B(red_op_B),
        .bypass_A(bypass_A), .bypass_B(bypass_B),
        .direction(direction),
        .out(out), .leds(leds)
    );

    initial clk = 0;
    always #5 clk = ~clk;

    integer i;
    reg [5:0] expected;

    initial begin
        // init
        rst = 1; A = 0; B = 0; opcode = 0; cin = 0;
        serial_in = 0; red_op_A = 0; red_op_B = 0;
        bypass_A = 0; bypass_B = 0; direction = 0;

        // 2.1 async reset check
        @(negedge clk);
        if (out !== 0 || leds !== 0)
            $display("Error: async reset failed");
        else
            $display("Pass: async reset");

        // 2.2 bypass
        bypass_A = 1; bypass_B = 1; rst = 0;
        repeat(20) begin
            A = $random; B = $random;
            opcode = $urandom_range(0, 5);
            repeat(2) @(negedge clk);
            if (out !== {3'b0, A})
                $display("Error: bypass A&B - out=%0h exp=%0h", out, {3'b0, A});
        end
        $display("Pass: bypass");

        // 2.3 opcode 0 AND
```

```
        bypass_A = 0; bypass_B = 0; opcode = 0;
        repeat(20) begin
            A = $random; B = $random;
            red_op_A = $random; red_op_B = $random;
            repeat(2) @(negedge clk);
            case ({red_op_A, red_op_B})
                2'b00: expected = {3'b0, A & B};
                2'b01: expected = {5'b0, &B};
                2'b10: expected = {5'b0, &A};
                2'b11: expected = {5'b0, &A};
            endcase
            if (out !== expected)
                $display("Error: AND - out=%0b exp=%0b", out, expected);
        end
        red_op_A = 0; red_op_B = 0;
        $display("Pass: opcode 0 AND");

        // 2.4 opcode 1 XOR
        opcode = 1;
        repeat(20) begin
            A = $random; B = $random;
            red_op_A = $random; red_op_B = $random;
            repeat(2) @(negedge clk);
            case ({red_op_A, red_op_B})
                2'b00: expected = {3'b0, A ^ B};
                2'b01: expected = {5'b0, ^B};
                2'b10: expected = {5'b0, ^A};
                2'b11: expected = {5'b0, ^A};
            endcase
            if (out !== expected)
                $display("Error: XOR - out=%0b exp=%0b", out, expected);
        end
        red_op_A = 0; red_op_B = 0;
        $display("Pass: opcode 1 XOR");

        // 2.5 opcode 2 Addition
        opcode = 2; red_op_A = 0; red_op_B = 0;
        repeat(20) begin
            A = $random; B = $random; cin = $random;
            repeat(2) @(negedge clk);
            if (out !== (A + B + cin))
                $display("Error: ADD - out=%0d exp=%0d", out, A+B+cin);
        end
        $display("Pass: opcode 2 Addition");
```

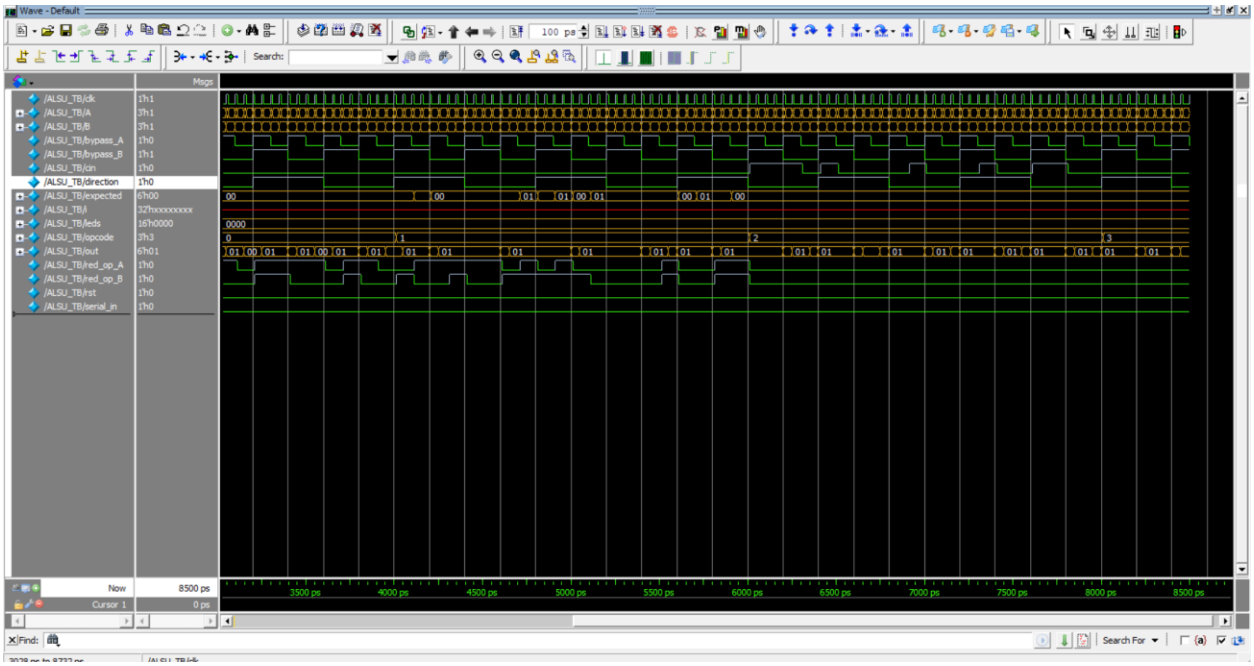
```
        // 2.6 opcode 3 Multiplication
        opcode = 3;
        repeat(20) begin
            A = $random; B = $random;
            repeat(2) @(negedge clk);
            if (out !== (A * B))
                $display("Error: MUL - out=%0d exp=%0d", out, A*B);
        end
        $display("Pass: opcode 3 Multiplication");

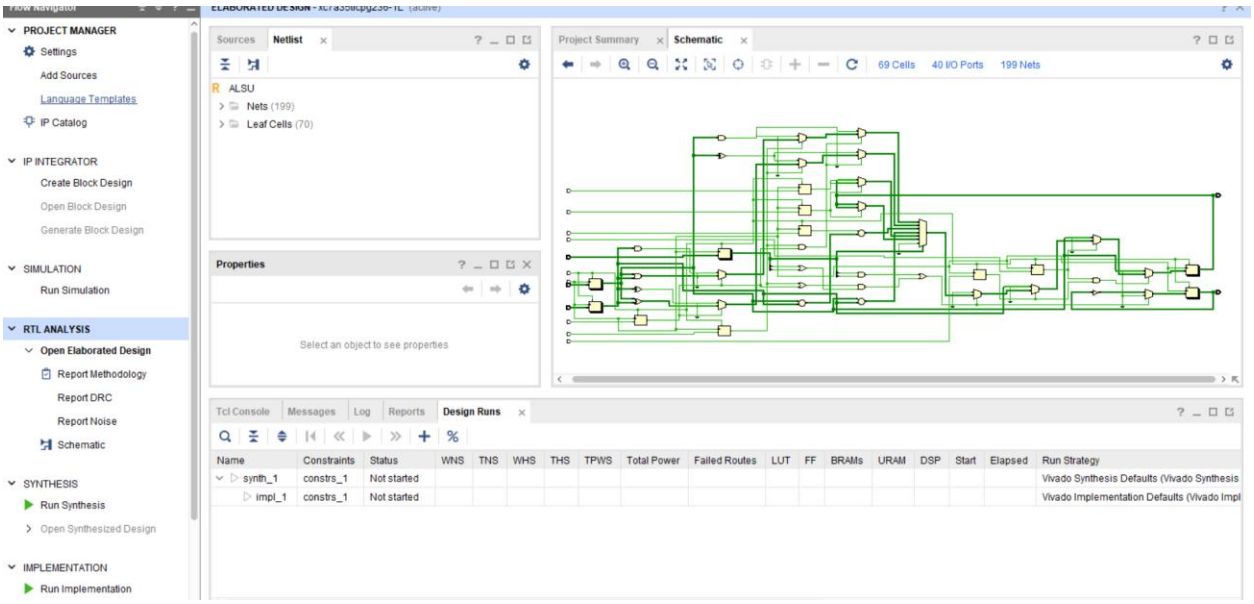
        // 2.7 opcode 4 Shift - check waveform
        opcode = 4;
        repeat(20) begin
            A = $random; B = $random;
            serial_in = $random; direction = $random;
            repeat(2) @(negedge clk);
        end
        $display("Check waveform for opcode 4 shift");

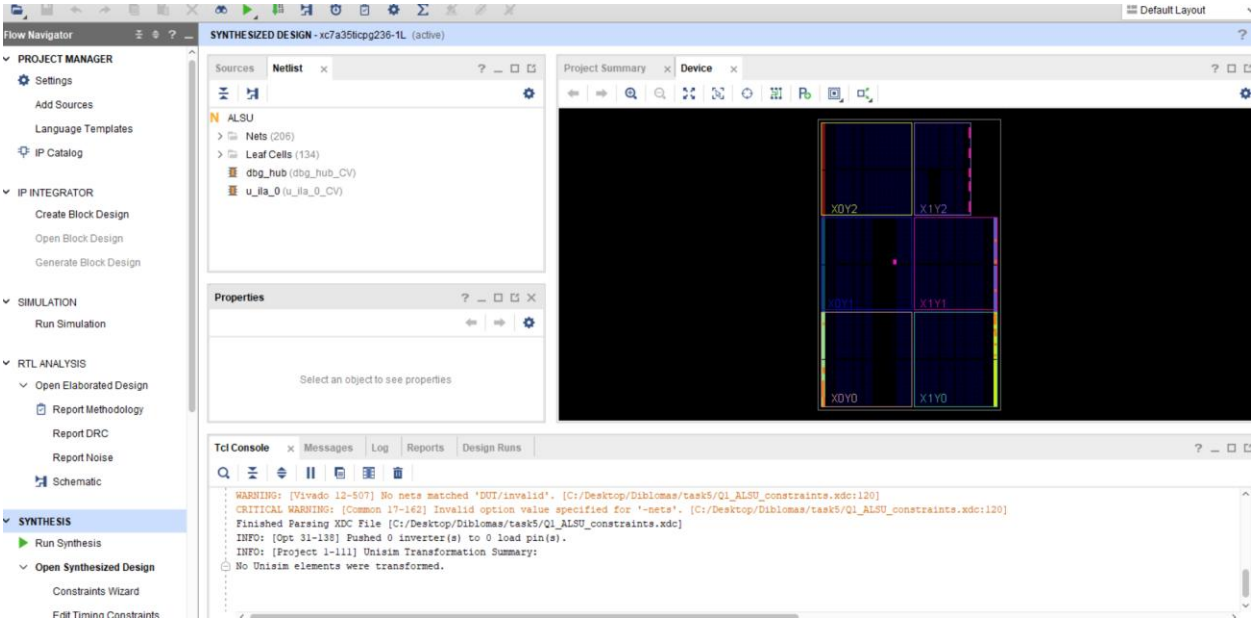
        // 2.8 opcode 5 Rotate - check waveform
        opcode = 5;
        repeat(20) begin
            A = $random; B = $random;
            serial_in = $random; direction = $random;
            repeat(2) @(negedge clk);
        end
        $display("Check waveform for opcode 5 rotate");

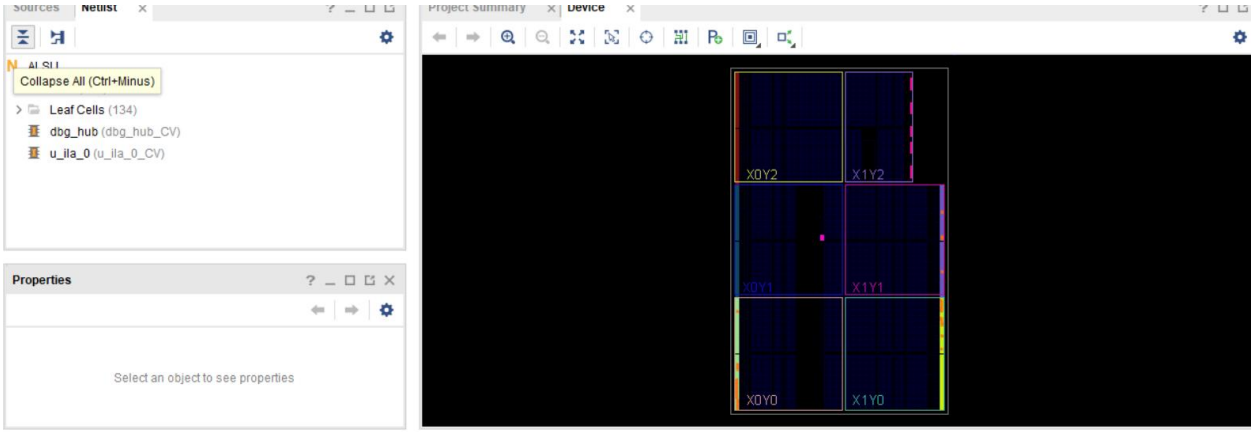
        $stop;
    end

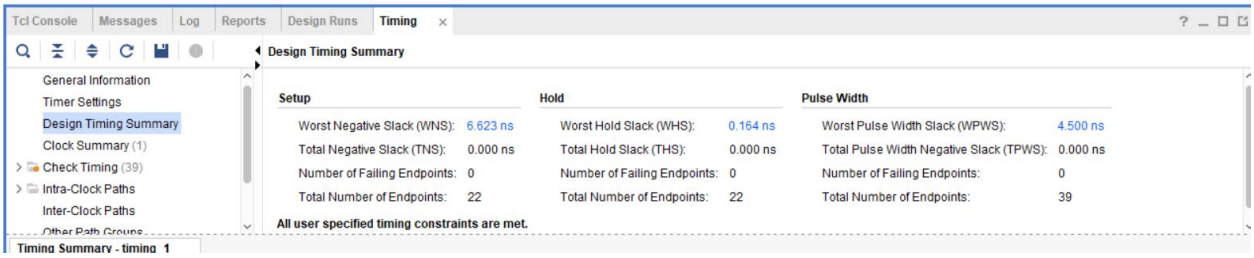
endmodule
```

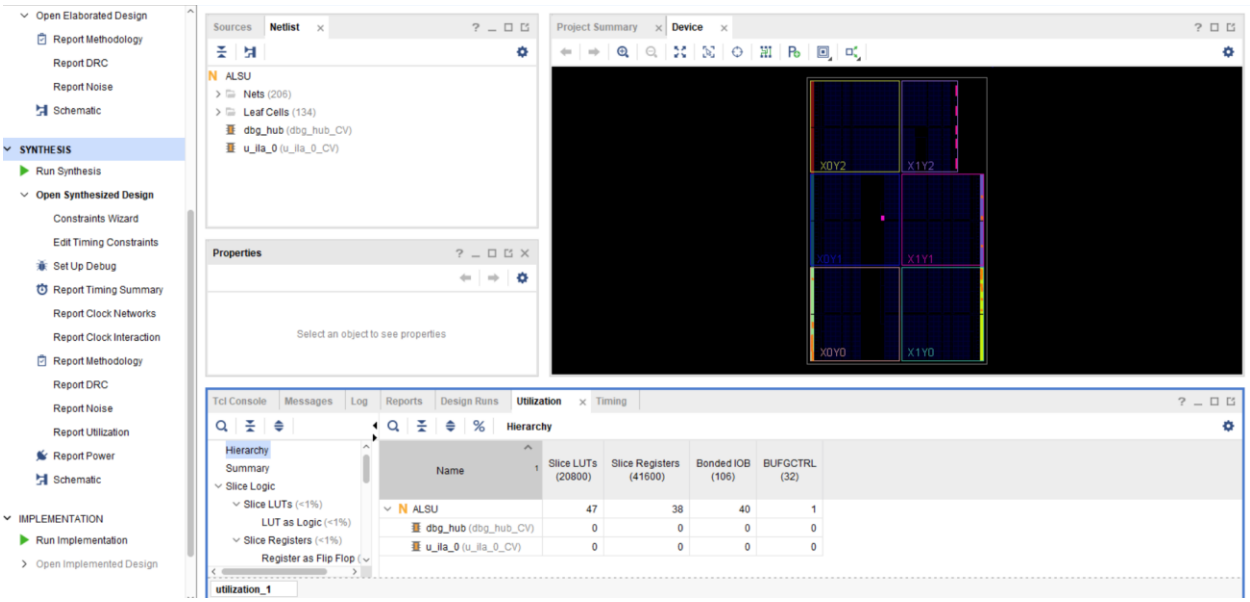

Wave - Default
/ALSU_TB/clk
/ALSU_TB/A
/ALSU_TB/B
/ALSU_TB/bypass_A
/ALSU_TB/bypass_B
/ALSU_TB/cin
/ALSU_TB/direction
/ALSU_TB/expected
/ALSU_TB/i
/ALSU_TB/leds
/ALSU_TB/opcode
/ALSU_TB/out
/ALSU_TB/red_op_A
/ALSU_TB/red_op_B
/ALSU_TB/rst
/ALSU_TB/serial_in
Now
8500 ps
Cursor 1
0 ps
3500 ps
4000 ps
4500 ps
5000 ps
5500 ps
6000 ps
6500 ps
7000 ps
7500 ps
8000 ps
8500 ps

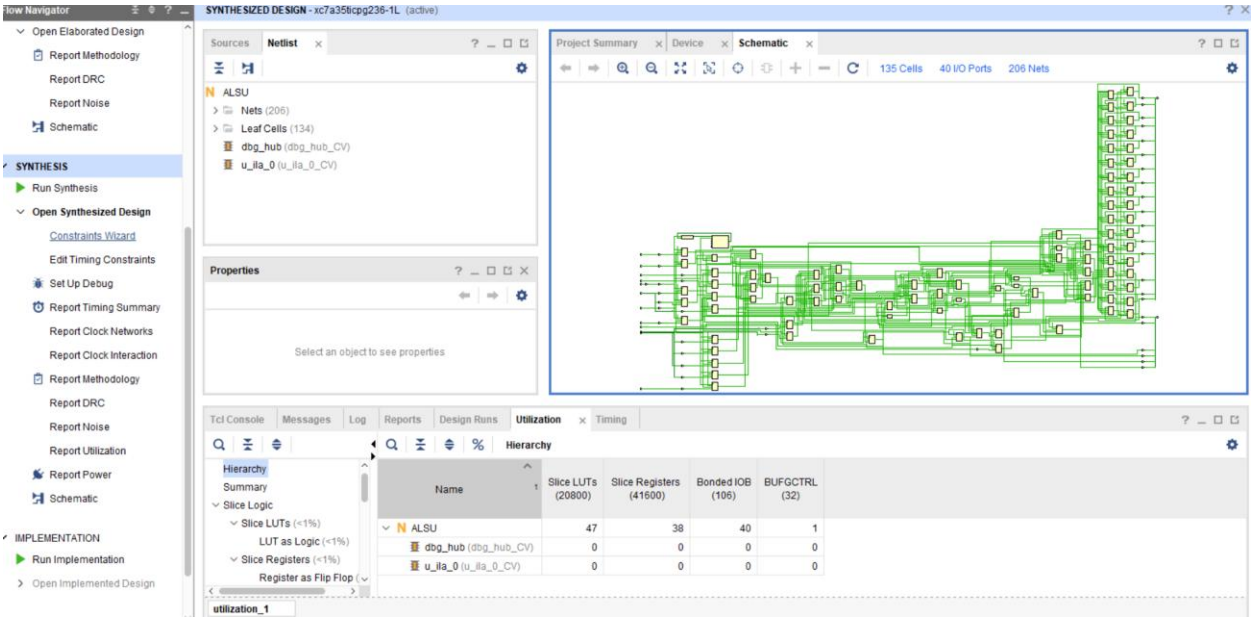

PROJECT MANAGER
Settings
Add Sources
Language Templates
IP Catalog
IP INTEGRATOR
Create Block Design
Open Block Design
Generate Block Design
SIMULATION
Run Simulation
RTL ANALYSIS
Open Elaborated Design
Report Methodology
Report DRC
Report Noise
Schematic
SYNTHESIS
Run Synthesis
Open Synthesized Design
IMPLEMENTATION
Run Implementation
Sources
Netlist
ALSU
Nets (199)
Leaf Cells (70)
Properties
Select an object to see properties
Project Summary
Schematic
69 Cells
40 I/O Ports
199 Nets
Tcl Console
Messages
Log
Reports
Design Runs
Name
Constraints
Status
WNS
TNS
WHS
THS
TPWS
Total Power
Failed Routes
LUT
FF
BRAMs
URAM
DSP
Start
Elapsed
Run Strategy
synth_1
constrs_1
Not started
Vivado Synthesis Defaults (Vivado Synthesis
impl_1
constrs_1
Not started
Vivado Implementation Defaults (Vivado Impl


Default Layout
Flow Navigator
SYNTHESIZED DESIGN - xc7a35tcpg236-1L (active)
PROJECT MANAGER
Settings
Add Sources
Language Templates
IP Catalog
IP INTEGRATOR
Create Block Design
Open Block Design
Generate Block Design
SIMULATION
Run Simulation
RTL ANALYSIS
Open Elaborated Design
Report Methodology
Report DRC
Report Noise
Schematic
SYNTHESIS
Run Synthesis
Open Synthesized Design
Constraints Wizard
Edit Timing Constraints
Sources
Netlist
ALSU
Nets (206)
Leaf Cells (134)
dbg_hub (dbg_hub_CV)
u_ila_0 (u_ila_0_CV)
Properties
Select an object to see properties
Project Summary
Device
X0Y2
X1Y2
X0Y1
X1Y1
X0Y0
X1Y0
Tcl Console
Messages
Log
Reports
Design Runs
WARNING: [Vivado 12-507] No nets matched 'DUT/invalid'. [C:/Desktop/Diblomas/task5/Q1_ALSU_constraints.xdc:120]
CRITICAL WARNING: [Common 17-162] Invalid option value specified for '-nets'. [C:/Desktop/Diblomas/task5/Q1_ALSU_constraints.xdc:120]
Finished Parsing XDC File [C:/Desktop/Diblomas/task5/Q1_ALSU_constraints.xdc]
INFO: [Opt 31-138] Pushed 0 inverter(s) to 0 load pin(s).
INFO: [Project 1-111] Unisim Transformation Summary:
No Unisim elements were transformed.


Sources
Netlist
Collapse All (Ctrl+Minus)
Leaf Cells (134)
dbg_hub (dbg_hub_CV)
u_ila_0 (u_ila_0_CV)
Properties
Select an object to see properties
Project Summary
Device
X0Y2
X1Y2
X0Y1
X1Y1
X0Y0
X1Y0


Tcl Console
Messages
Log
Reports
Design Runs
Timing
Design Timing Summary
General Information
Timer Settings
Design Timing Summary
Clock Summary (1)
Check Timing (39)
Intra-Clock Paths
Inter-Clock Paths
Setup
Worst Negative Slack (WNS): 6.623 ns
Total Negative Slack (TNS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 22
Hold
Worst Hold Slack (WHS): 0.164 ns
Total Hold Slack (THS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 22
Pulse Width
Worst Pulse Width Slack (WPWS): 4.500 ns
Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 39
All user specified timing constraints are met.
Timing Summary - timing_1





Q2-

```
`timescale 1ns/1ps

module posedge_detector (
    input  wire clk, rst_n,
```

```
    input  wire sig_in,
    output wire pulse_out
);
    reg sig_d;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) sig_d <= 0;
        else        sig_d <= sig_in;
    end

    assign pulse_out = sig_in & ~sig_d;
endmodule


module cdc_sync_bit #(parameter STAGES = 2)(
    input  wire clk, rst_n,
    input  wire d_async,
    output wire q_sync
);
    integer i;
    reg [STAGES-1:0] chain;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) chain <= 0;
        else begin
            chain[0] <= d_async;
            for (i = 1; i < STAGES; i = i + 1)
                chain[i] <= chain[i-1];
        end
    end

    assign q_sync = chain[STAGES-1];
endmodule


module cdc_pulse_sync #(parameter STAGES = 2)(
    input  wire src_clk, src_rst_n,
    input  wire src_pulse,
    input  wire dst_clk, dst_rst_n,
    output wire dst_pulse
);
    reg src_toggle;
    integer i;
    reg [STAGES:0] dst_chain;
```

```
    always @(posedge src_clk or negedge src_rst_n) begin
        if (!src_rst_n)     src_toggle <= 0;
        else if (src_pulse) src_toggle <= ~src_toggle;
    end

    always @(posedge dst_clk or negedge dst_rst_n) begin
        if (!dst_rst_n) dst_chain <= 0;
        else begin
            dst_chain[0] <= src_toggle;
            for (i = 1; i <= STAGES; i = i + 1)
                dst_chain[i] <= dst_chain[i-1];
        end
    end

    assign dst_pulse = dst_chain[STAGES] ^ dst_chain[STAGES-1];
endmodule
```

```
`timescale 1ns/1ps

module mini_soc (
    input wire clk_ctrl,
    input wire clk_wrk,
    input wire rst_n
);
    wire start_pulse_ctrl, start_pulse_wrk;
    wire cmd_valid_ctrl, cmd_valid_wrk, cmd_valid_synced;
    wire cmd_ready;
    wire [7:0] cmd_data_ctrl, cmd_data_wrk;
    wire done_pulse_wrk, done_pulse_ctrl;

    assign cmd_ready    = 1'b1;
    assign cmd_data_wrk = cmd_data_ctrl;

    // Problem 1: start_pulse fast->slow using pulse synchronizer
    cdc_pulse_sync #(.STAGES(2)) u_pulse_sync (
        .src_clk   (clk_ctrl),
        .src_rst_n (rst_n),
        .src_pulse (start_pulse_ctrl),
        .dst_clk   (clk_wrk),
        .dst_rst_n (rst_n),
```

```
        .dst_pulse (start_pulse_wrk)
    );

    // Problem 2 step 1: sync cmd_valid level signal with 2-FF
    cdc_sync_bit #(.STAGES(2)) u_bit_sync (
        .clk     (clk_wrk),
        .rst_n   (rst_n),
        .d_async (cmd_valid_ctrl),
        .q_sync  (cmd_valid_synced)
    );

    // Problem 2 step 2: convert level to pulse using posedge detector
    posedge_detector u_pos_detect (
        .clk      (clk_wrk),
        .rst_n    (rst_n),
        .sig_in   (cmd_valid_synced),
        .pulse_out(cmd_valid_wrk)
    );

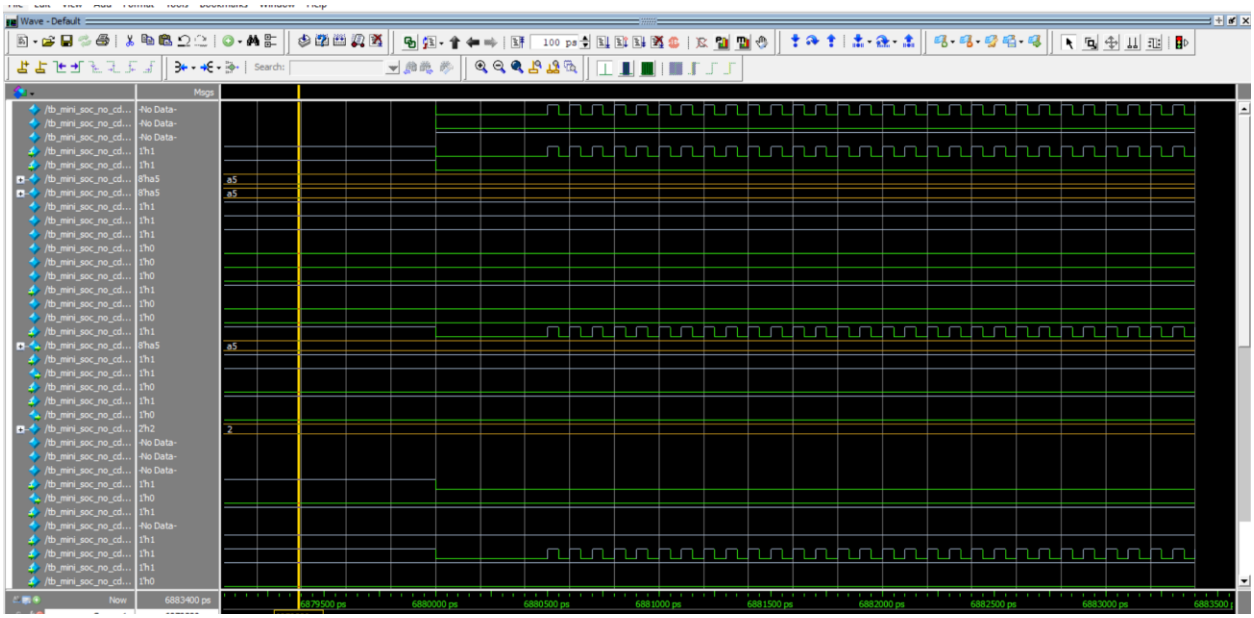
    // Problem 3: done_pulse slow->fast using pulse synchronizer
    cdc_pulse_sync #(.STAGES(2)) u_pulse_sync2 (
        .src_clk   (clk_wrk),
        .src_rst_n (rst_n),
        .src_pulse (done_pulse_wrk),
        .dst_clk   (clk_ctrl),
        .dst_rst_n (rst_n),
        .dst_pulse (done_pulse_ctrl)
    );

    ctrl_ip u_ctrl (
        .clk        (clk_ctrl),
        .rst_n      (rst_n),
        .start_pulse(start_pulse_ctrl),
        .cmd_valid  (cmd_valid_ctrl),
        .cmd_data   (cmd_data_ctrl),
        .cmd_ready  (cmd_ready),
        .done_pulse (done_pulse_ctrl)
    );

    worker_ip u_worker (
        .clk        (clk_wrk),
        .rst_n      (rst_n),
        .start_pulse(start_pulse_wrk),
        .cmd_valid  (cmd_valid_wrk),
        .cmd_data   (cmd_data_wrk),
```

```
        .done_pulse (done_pulse_wrk)
    );

endmodule
```



Q3-

```
`timescale 1ns/1ps

module binary_to_gray #(parameter WIDTH = 8)(
    input  wire [WIDTH-1:0] binary,
    output wire [WIDTH-1:0] gray
);
    assign gray[WIDTH-1] = binary[WIDTH-1];

    genvar i;
    generate
        for (i = WIDTH-2; i >= 0; i = i - 1) begin : b2g
            assign gray[i] = binary[i+1] ^ binary[i];
        end
    endgenerate
```

```
endmodule


module gray_to_binary #(parameter WIDTH = 8)(
    input  wire [WIDTH-1:0] gray,
    output wire [WIDTH-1:0] binary
);
    assign binary[WIDTH-1] = gray[WIDTH-1];

    genvar i;
    generate
        for (i = WIDTH-2; i >= 0; i = i - 1) begin : g2b
            assign binary[i] = binary[i+1] ^ gray[i];
        end
    endgenerate
endmodule


module DFF2_sync #(
    parameter WIDTH   = 8,
    parameter RSTTYPE = "SYNC",
    parameter STAGES  = 2
)(
    input  wire              clk, rst,
    input  wire [WIDTH-1:0]  d_async,
    output wire [WIDTH-1:0]  q_sync
);
    integer i;
    reg [WIDTH-1:0] chain [0:STAGES-1];

    generate
        if (RSTTYPE == "SYNC") begin : sync_rst
            always @(posedge clk) begin
                if (rst) begin
                    for (i = 0; i < STAGES; i = i + 1) chain[i] <= 0;
                end else begin
                    chain[0] <= d_async;
                    for (i = 1; i < STAGES; i = i + 1) chain[i] <= chain[i-1];
                end
            end
        end else begin : async_rst
            always @(posedge clk or posedge rst) begin
                if (rst) begin
                    for (i = 0; i < STAGES; i = i + 1) chain[i] <= 0;
                end else begin
```

```
                    chain[0] <= d_async;
                    for (i = 1; i < STAGES; i = i + 1) chain[i] <= chain[i-1];
                end
            end
        end
    endgenerate

    assign q_sync = chain[STAGES-1];
endmodule


module gray_enc_sync #(
    parameter WIDTH   = 8,
    parameter RSTTYPE = "SYNC"
)(
    input  wire [WIDTH-1:0] tx_data,
    output wire [WIDTH-1:0] rx_data,
    input  wire             tx_clk, rx_clk, rst
);
    reg  [WIDTH-1:0] tx_data_r, gray_r;
    wire [WIDTH-1:0] gray_w, gray_synced;

    binary_to_gray #(.WIDTH(WIDTH)) u_b2g (.binary(tx_data_r), .gray(gray_w));

    generate
        if (RSTTYPE == "SYNC") begin : tx_sync
            always @(posedge tx_clk) begin
                if (rst) begin tx_data_r <= 0; gray_r <= 0; end
                else     begin tx_data_r <= tx_data; gray_r <= gray_w; end
            end
        end else begin : tx_async
            always @(posedge tx_clk or posedge rst) begin
                if (rst) begin tx_data_r <= 0; gray_r <= 0; end
                else     begin tx_data_r <= tx_data; gray_r <= gray_w; end
            end
        end
    endgenerate

    // one 2-FF sync per gray bit
    genvar b;
    generate
        for (b = 0; b < WIDTH; b = b + 1) begin : cdc_bits
            DFF2_sync #(.WIDTH(1), .RSTTYPE(RSTTYPE)) u_sync (
                .clk(rx_clk), .rst(rst),
                .d_async(gray_r[b]),
```

```
                .q_sync(gray_synced[b])
            );
        end
    endgenerate

    gray_to_binary #(.WIDTH(WIDTH)) u_g2b (.gray(gray_synced), .binary(rx_data));

endmodule
```

Tb-

```
`timescale 1ns/1ps

module gray_enc_sync_TB;

    parameter WIDTH = 8;

    reg  [WIDTH-1:0] tx_data;
    wire [WIDTH-1:0] rx_data;
    reg  tx_clk, rx_clk, rst;

    gray_enc_sync #(.WIDTH(WIDTH), .RSTTYPE("SYNC")) DUT (
        .tx_data(tx_data), .rx_data(rx_data),
        .tx_clk(tx_clk), .rx_clk(rx_clk), .rst(rst)
    );

    initial tx_clk = 0;
    always #5  tx_clk = ~tx_clk;

    initial rx_clk = 0;
    always #20 rx_clk = ~rx_clk;

    integer i;

    initial begin
        tx_data = 0; rst = 1;
        repeat(4) @(posedge tx_clk);
```

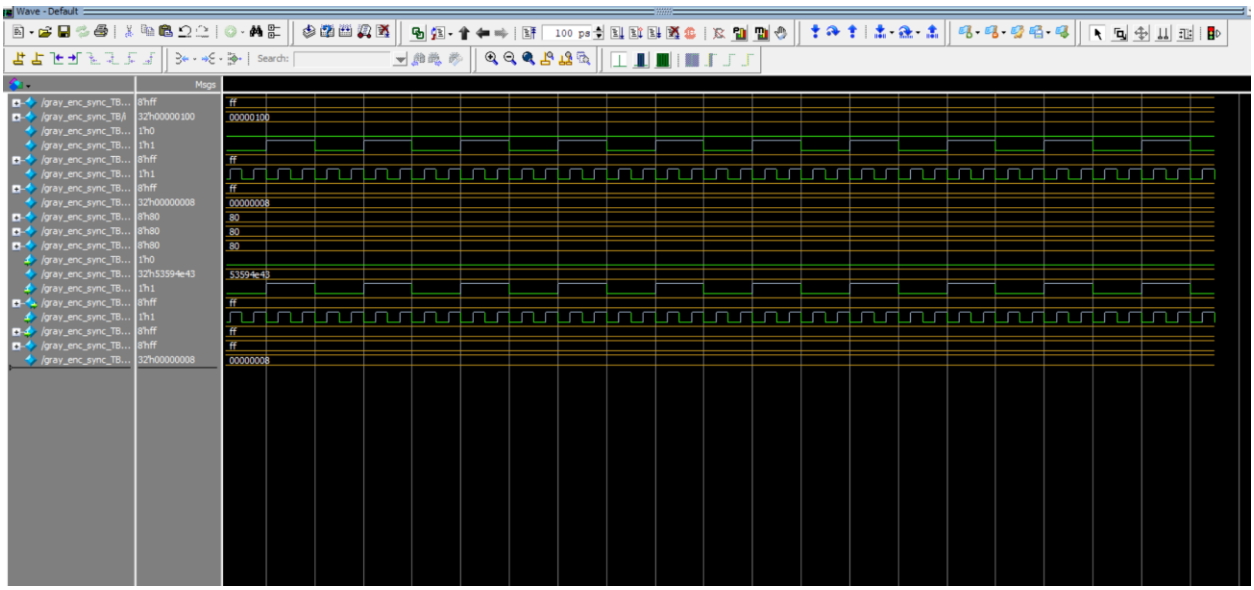
```
        @(posedge rx_clk);
        rst = 0;

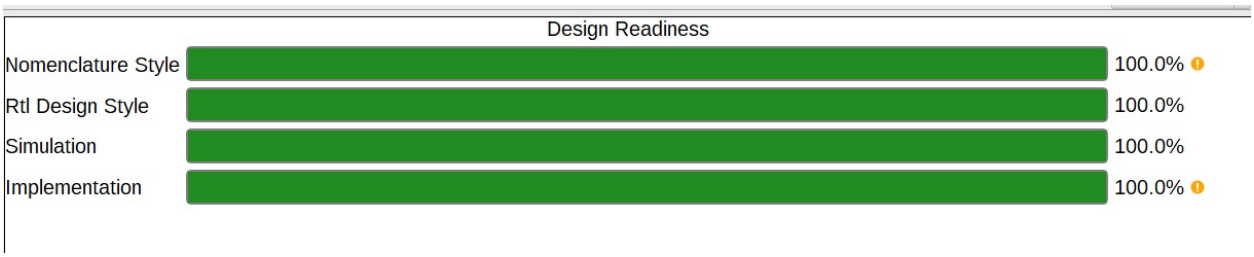
        @(negedge rx_clk);
        if (rx_data !== 0)
            $display("Error: reset - rx_data = %0h", rx_data);
        else
            $display("Pass: reset");

        // drive sequential values and check output after pipeline settles
        for (i = 0; i < 256; i = i + 1) begin
            tx_data = i[WIDTH-1:0];
            repeat(5) @(posedge rx_clk);
            @(negedge rx_clk);
            if (rx_data !== tx_data)
                $display("Error: tx=%0d rx=%0d", tx_data, rx_data);
        end
        $display("Pass: all values checked");

        $stop;
    end

endmodule
```




Design Readiness
Nomenclature Style
100.0%
Rtl Design Style
100.0%
Simulation
100.0%
Implementation
100.0%


Fixed
Pending
Verified
Total : 1
Selected : 0
Alias
Message
Module
Category
State
Owne
always_signal_assign_large
Always block has more signal as
Q3_gray_enc
Rtl Design Style
open
unassi